# Prvi praktični test iz
# Algoritama i struktura podataka

Prvi praktični test održan 26.12.2025.

Ime i prezime: _____

Broj indeksa: _____

## Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test1_GrupaF_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**
9. **Svaki zadatak nosi po 8 bodova. Kod za testiranje algoritama u Main metodi donosi još jedan bod.**

Pre početka testa napišite ime, prezime i broj indeksa.

**PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!**

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

## Zadatak 1 – *Quick Sort*

Napisati ***quickSort*** metodu ***Quick Sort*** algoritma za sortiranje celobrojnog niza dužine ***n***.

Dodatne napomene za rešavanje zadatka:

- Ulaz je nesortiran niz celih brojeva
- Izlaz je sortirani niz celih brojeva
- U template-u se nalazi klasa ***RandomArrayGenerator*** koju treba iskoristiti za generisanje niza slučajnih brojeva
- U okviru klase ***QuickSort*** napisati implementaciju za metodu ***private static void quickSort(int[] arr, int lower, int upper)***
- Obezbediti kod u **Main** metodi za testiranje algoritma.

## Zadatak 2 – *Pronalaženje najduže podliste parnih brojeva*

Napisati algoritam koji pronalazi najdužu podlistu parnih elemenata. Vrednosti iz pronađene podliste ispisati u konzoli, ili da takva podlista ne postoji.

Dodatne napomene za rešavanje zadatka:

- Zahtevan potpis metode: ***public static void findLargestEvenNumbersSubList()***
- Metodu pisati u okviru klase ***LinkedList***

Primer:

- Input: ***LinkedList: 1 -> 0 -> 6 -> 8 -> 4***
- Output: ***0 6 8 4***

- Input: ***LinkedList: 1 -> 5 -> 3***
- Output: ***Ne postoji podlista parnih brojeva***

## Zadatak 3 – *Hot Potato Game – Odd and Even Problem*

**Problem:** U klasičnoj igri "Vrući krompir", igrači u krugu prebacuju krompir dok muzika svira. Kada muzika stane, igrač koji drži krompir ispada. U našoj **naprednoj varijanti**, broj prolazaka krompira se dinamički menja tokom igre.

**Pravila igre:**

1. Ima n igrača numerisanih od 1 do n
2. Početni broj prolazaka je K
3. **Pravilo eliminacije:** U svakom krugu, krompir se prosledi K puta. Igrač kod koga se krompir nađe nakon **K**-tog prolaska ispada.
4. **Dinamička promena K:** Nakon što igrač ispadne, novi broj prolazaka ($\mathbf{K_{novi}}$) za sledeći krug se računa na osnovu ID-a eliminisanog igrača:

   - Ako je ID ispalog igrača **paran** broj: $K_{novi} = K_{stari} + 2$
   - Ako je ID ispalog igrača **neparan** broj: $K_{novi} = K_{stari} - 1$

5. **Ograničenja za K:** * Ako **K** postane manji od 1, postavlja se na vrednost početnog **K** (iz parametra metode).

- Pre nego što počne novi krug, ako je **K** veći od broja preostalih igrača, uzima se ostatak pri deljenju: $\boldsymbol{K = (K \% broj_preostalih) + 1}$

6. Igra se nastavlja dok ne ostane samo jedan igrač – **pobednik**.

<u>Dodatni zahtevi za rešavanje zadatka:</u>

1. Zadatak rešavati u okviru metode: ***public int PlayHotPotato(int totalPlayers, int initialPasses)***
2. Morate koristiti ***Queue*** za predstavljanje kruga igrača.
3. Implementirati dinamičko računanje prolazaka po datoj formuli.
4. Rezultat izvršavanja algoritma treba da bude redni broj pobednika u igri.

<u>Primer:</u>

| | | |
|---|---|---|
| Input 1: | totalPlayers = 4, | initialPasses = 2 |
| Output 1: | expectedWinner = 3 | |
| | | |
| Input 2: | totalPlayers = 6, | initialPasses = 1 |
| Output 2: | expectedWinner = 6 | |